

Compte Rendu de Travaux Pratiques

Formation : M2 SETI / Systèmes embarqués et traitement de l'information
Matière : C3 - Intelligence Artificielle pour la robotique

TP réalisé à l'UPSay - Université Paris-Saclay, le 9 janvier 2026.

Table des matières

1 Localisation	3
1.1 Estimer la position	3
1.1.1 Calculer le déplacement de chaque roue du robot	3
1.1.2 Vérification de la cohérence des résultats par rapport au réel	3
1.1.3 Programmation du modèle	4
1.1.4 Vérification des résultats	4
1.2 Tracer la trajectoire	8
1.2.1 Tracer en temps réel la trajectoire suivie	8
1.2.2 Ajout des murs	9
1.2.3 Tracer la trajectoire réelle du robot	10
1.2.4 Tracer l'évolution des valeurs réelles et estimées	11
1.2.5 Qualifier l'évolution de la précision sur la localisation	12
2 Déplacements automatisés	13
2.1 Points de destination	13
2.2 Changement de repère	13
2.3 Asservissement	14

Chapitre 1

Localisation

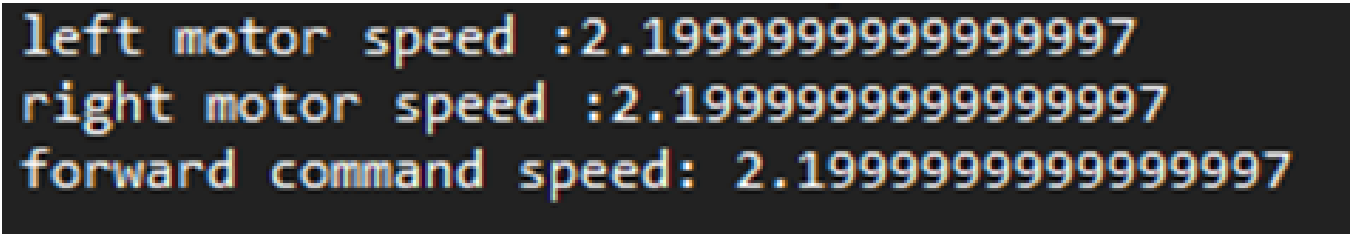
1.1 Estimer la position

1.1.1 Calculer le déplacement de chaque roue du robot

Dans cette première partie, j'ai utilisé le simulateur pour calculer le déplacement de chaque roue du robot à chaque t par rapport au précédent en utilisant l'api fournie. Voici le programme permettant d'obtenir la vitesse sur les deux roues, ainsi que la distance linéaire parcourue :

```
1 v_l = motor_left.getVelocity()
2 v_r = motor_right.getVelocity()
3
4
5 delta_l = v_l * r * delta_t
6 delta_r = v_r * r * delta_t
7
```

1.1.2 Vérification de la cohérence des résultats par rapport au réel



```
left motor speed :2.1999999999999997
right motor speed :2.1999999999999997
forward command speed: 2.1999999999999997
```

Figure 1.1 – Capture d'écran de la console de Webots, affichant des données clés.

Cela me permet de vérifier en ligne droite que les deux roues tournent à la même vitesse. Lors d'un virage, les deux vitesses sont différentes. Les résultats seront vérifiés par la suite.

1.1.3 Programmation du modèle

En sachant que le rayon de la roue du thymio est de 2.1cm et que l'écartement entre ses roues est d'environ 10.8cm, je peux utiliser le modèle de déplacement d'un tricycle holonome. On omet de tenir compte de l'usure et de la pression/adhérence des roues dans cette simulation.

$$\begin{pmatrix} \delta s \\ \delta \theta \end{pmatrix} = \begin{pmatrix} \frac{\delta_r + \delta_l}{2}, \frac{\delta_l - \delta_r}{2e} \end{pmatrix}$$
$$(\mathbf{x}_{k+1})_{R_{glob}} = \begin{pmatrix} x_k + \delta s \cos(\theta_k + \frac{\delta \theta}{2}) \\ z_k + \delta s \sin(\theta_k + \frac{\delta \theta}{2}) \\ \theta_k + \delta \theta \end{pmatrix}$$

Figure 1.2 – modèle de déplacement d'un robot holonome

En utilisant les distances linéaires des roues gauche et droite, je peux utiliser le modèle holonome pour estimer une position (x,y) en un plan 2D. Le programme python ci-dessous en est une implémentation :

```
1
2 delta_s = (delta_r + delta_l) / 2.0
3 delta_theta = (delta_l - delta_r) / (2.0 * e)
4
5 x = x + delta_s * math.cos(theta + delta_theta / 2.0)
6 y = y + delta_s * math.sin(theta + delta_theta / 2.0) theta = theta + delta_theta
7
```

Par l'ajout de la progression à chaque itération, la position (x, y) est mise à jour au fur et à mesure du parcours du robot.

1.1.4 Vérification des résultats

Pour vérifier les résultats, je teste la distance parcourue estimée par le robot sur une distance connue. Ici, nous savons que les tuiles font respectivement 25cm de large. Mon protocole de test consiste à démarrer le robot d'une position nulle, de le déplacer d'une tuile et de le faire revenir au point initial.

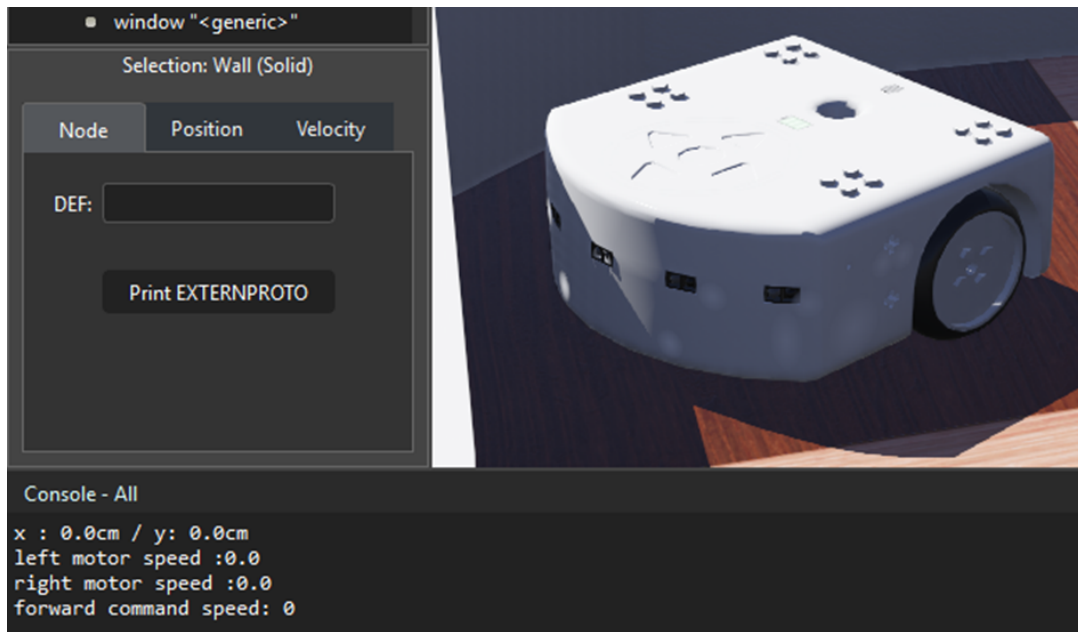


Figure 1.3 – Position par défaut du robot en $x=0$ et $y=0$.

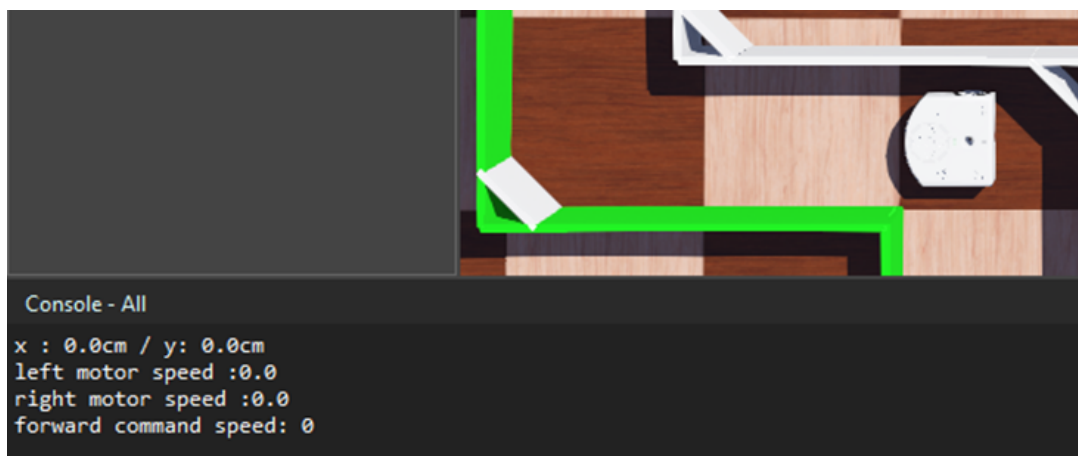


Figure 1.4 – Démarrage de la simulation avec le robot en position de départ

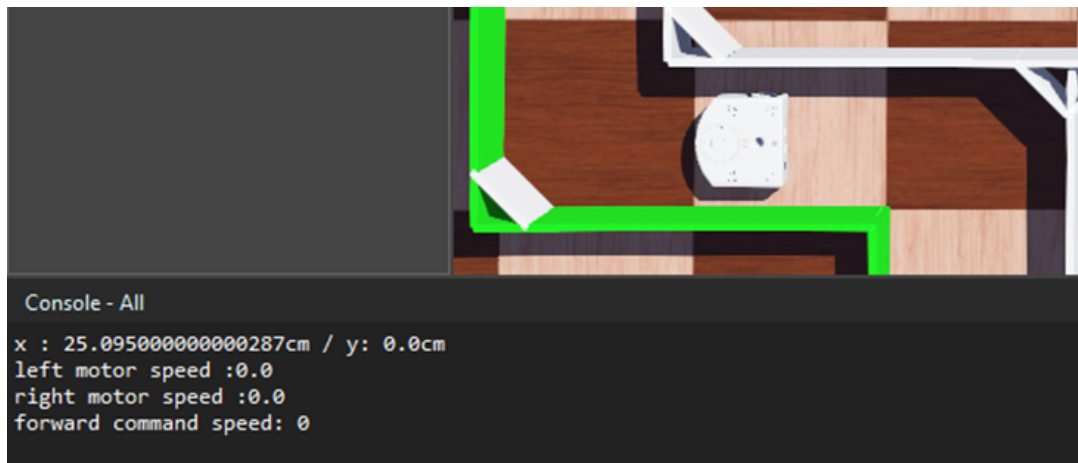


Figure 1.5 – Le robot parcourt une tuile de long, soit 25cm

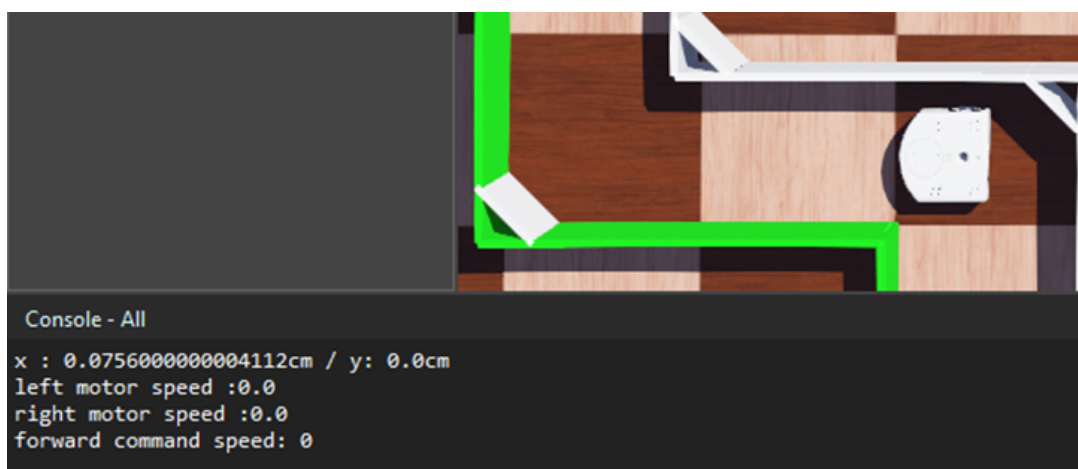


Figure 1.6 – Retour du robot au point de départ en marche arrière, il réalise -25cm

Durant la simulation, j'observe que l'estimation semble correcte, bien qu'un biais pourrait être observé dans le temps. On ajoute en erreur le contrôleur humain qui ne remet pas idéalement le robot dans sa position initiale. (environ 0.07cm d'erreur).

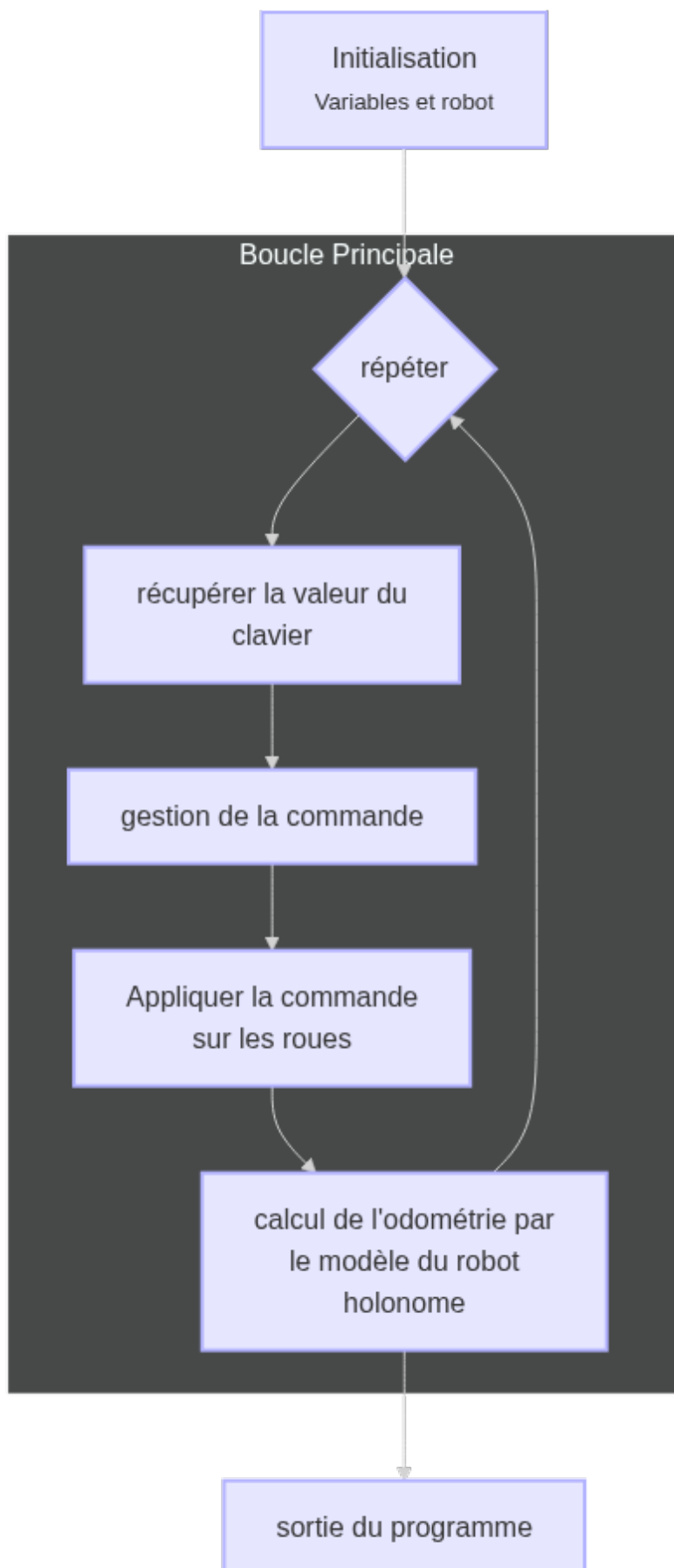


Figure 1.7 – Diagramme de séquence du programme

1.2 Tracer la trajectoire

1.2.1 Tracer en temps réel la trajectoire suivie

Pour tracer la trajectoire, j'utilise matplotlib, avec une mise à jour toutes les 10 itérations de boucle pour éviter de surcharger webots. Les coordonnées (x, y) estimées du robot sont accumulées dans une liste de points. Cette méthode est implémentée par le code suivant :

```
1
2 # dans la boucle while
3 plot_counter += 1
4 if plot_counter % 10 == 0:
5     plt.clf()
6     plt.plot(list_pos_x, list_pos_y, 'b-')
7
8     plt.plot(x, y, 'ro')
9
10    plt.draw()
11    plt.pause(0.001)
12
13
```

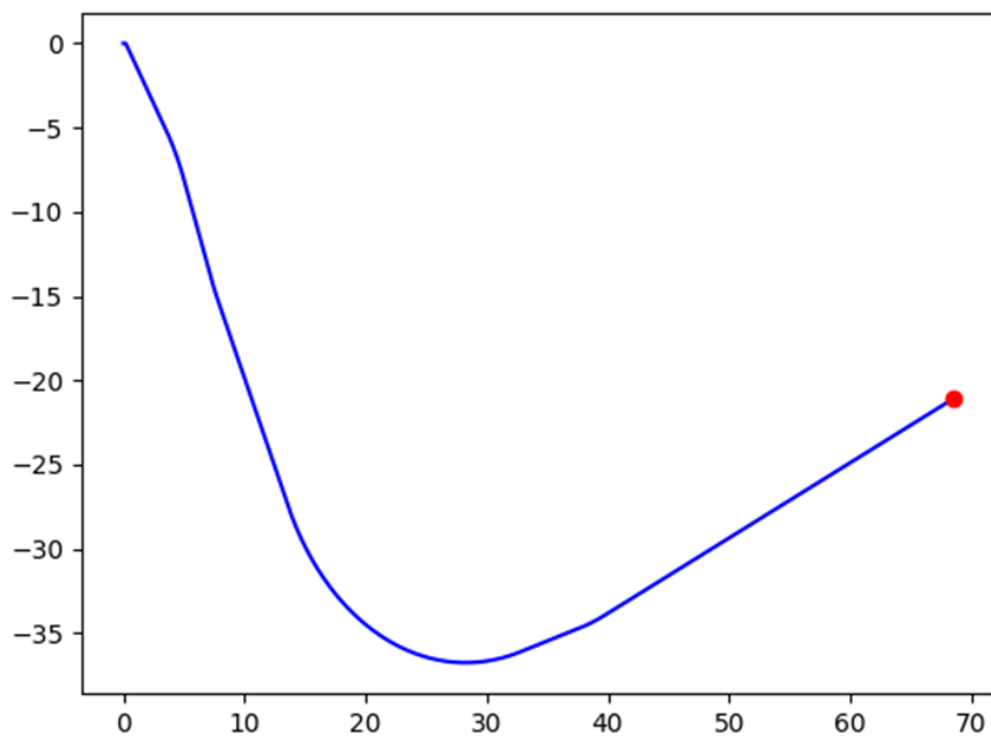


Figure 1.8 – Trajectoire obtenue pour thymio sur le graphique généré

1.2.2 Ajout des murs

Pour afficher les murs, j'ai récupéré le fichier du monde webots et extrait les différentes données de chaque mur du circuit. Ensuite, j'ai utilisé une fonction pour afficher les murs en rouge en créant des coins et en remplissant le rectangle donné. A chaque affichage du graphique, tous les murs doivent être retracés en même temps que la courbe. Cela est un process lourd et certainement optimisable.

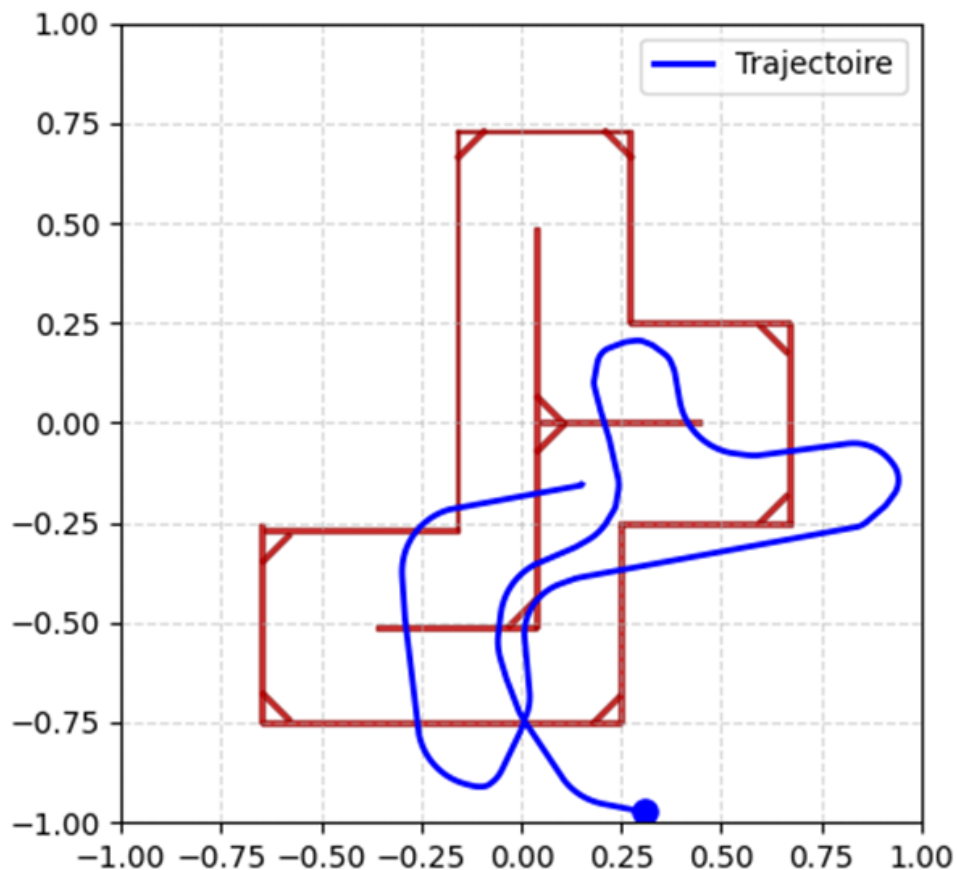


Figure 1.9 – Trajectoire estimée complète avec affichage du circuit

Sur l'image, on peut voir que la trajectoire du robot ne suit pas le circuit réel. Cela est dû à l'erreur de l'estimation de position angulaire au point de départ. Bien que la trajectoire dessinée ne soit pas dans le bon angle, on remarque que le point d'arrivée du robot ne correspond pas au point de départ. Cela est dû à l'erreur de mesure qui s'accumule. Cette erreur devient encore plus grande lorsque le robot percute un mur ou réalise une séquence de marche avant/arrière.

Code de la fonction de création des murs :

```
1
2 def get_rotated_rect_corners(center_pos, size, angle_rad):
3     cx, cy = center_pos
4     w, h = size[0] / 2.0, size[1] / 2.0 # Demi-largeur et demi-hauteur
5
6     #calcul des coins des murs par rapport aux coordonnées et tailles
7     corners_local = np.array([[ -w, -h], [w, -h], [w, h], [ -w, h]])
8
9     #position
10    c, s = np.cos(angle_rad), np.sin(angle_rad)
11    rot_matrix = np.array([[c, -s], [s, c]])
12
13    #appliquer la matrice de rotation sur les coins
14    rotated_corners = corners_local.dot(rot_matrix.T)
15
16    #ajout du centre
17    final_corners = rotated_corners + np.array([cx, cy])
18
19    return final_corners
20
21 # exemple de murs :
22 walls_config = [
23     {'pos': (-0.16, 0.23), 'size': (0.01, 1.0), 'angle': 0.0},
24
25     {'pos': (0.05, 0.73), 'size': (0.43, 0.01), 'angle': 0.0},
26 ]
```

1.2.3 Tracer la trajectoire réelle du robot

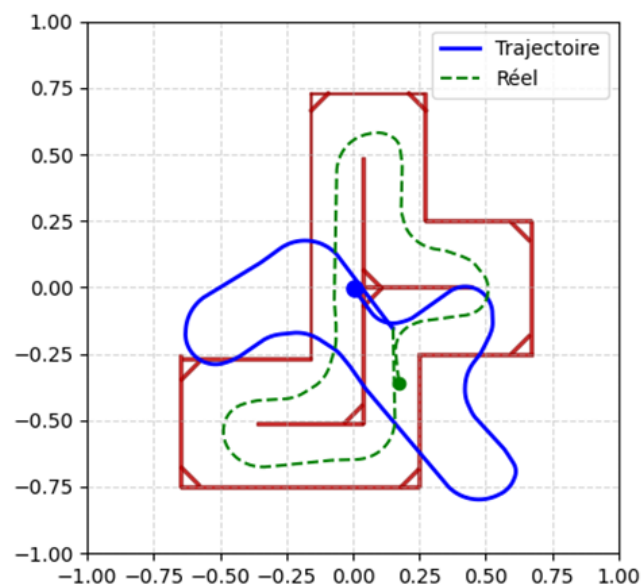


Figure 1.10 – Trajectoire estimée complète avec affichage de la trajectoire réelle

En utilisant le supervisor, on peut récupérer les coordonnées réelles du robot et les afficher sur notre graphique. Je remarque alors une trajectoire semblable mais avec un repère différent. Grace au supervisor, je sais que l'angle theta au départ vaut -2.615814835873021 et après avoir remarqué une erreur de signe dans mon calcul de theta, j'obtiens la trajectoire estimée suivante :

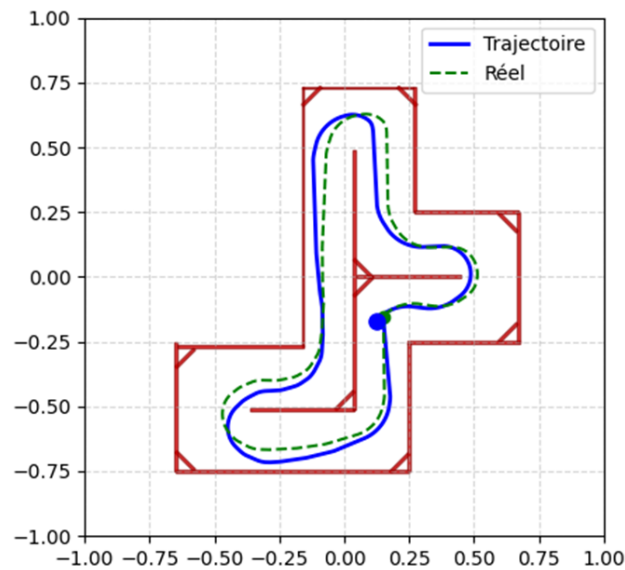


Figure 1.11 – Trajectoire estimée après correction de theta initial

1.2.4 Tracer l'évolution des valeurs réelles et estimées

Pour obtenir l'évolution des valeurs de x, y et theta, je stocke à chaque itération ces valeurs dans des listes. Ensuite, je réalise un subplot pour chaque valeur et je trace en fonction du temps. Les données réelles sont en vert et les données estimées sont en bleu. Le tout est réalisé à chaque itération et l'affichage mis à jour toutes les 10 itérations.

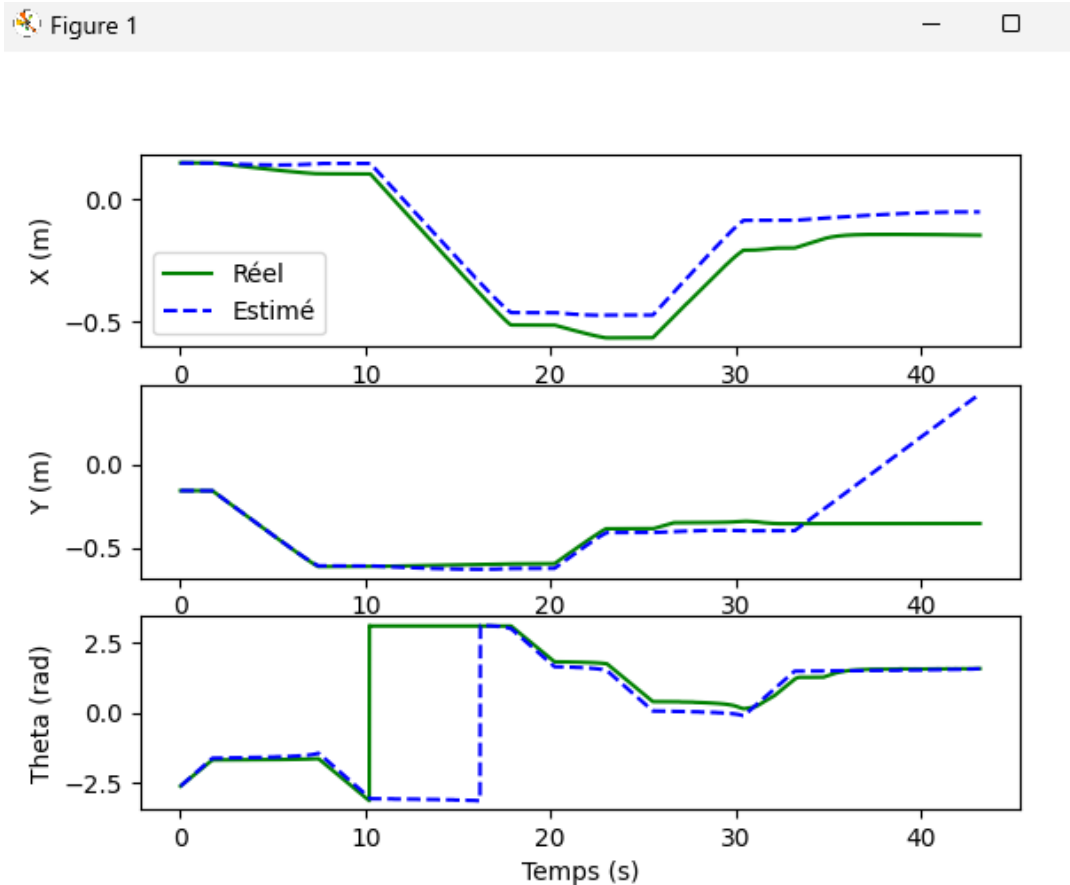


Figure 1.12 – Evolution des valeurs réelles et estimées

1.2.5 Qualifier l'évolution de la précision sur la localisation

L'objectif de cette étape était d'implémenter un modèle d'estimation d'état basé sur l'odométrie. En confrontant les valeurs calculées aux mesures du superviseur, j'ai pu optimiser le programme pour faire converger la position estimée vers la trajectoire réelle du robot.

Parmis les optimisations apportées, j'ai :

- Normalisé l'angle θ à chaque itération pour maintenir l'angle dans une intervalle $[-\pi, \pi]$,
- Placé le calcul de l'estimation au début de boucle pour évaluer la progression de l'itération précédente,
- Corrigé le sens de rotation par l'ajustement du repère du robot au repère global ($\theta = \theta - \theta_{\text{heta}}$),
- Ajouté l'angle moyen dans le déplacement du robot lorsque celui-ci tourne sur lui-même (éviter ainsi le fait de couper les virages) avec : $x = x + \delta_s \cos\left(\theta + \frac{\delta\theta}{2}\right)$

Par les graphiques d'évolution présentés ci-dessus, je remarque que l'estimation tend vers la valeur réelle mais un biais s'accumule de manière rapide au fil du temps.

Chapitre 2

Déplacements automatisés

2.1 Points de destination

Initialement, j'ai testé de créer un point cible de coordonnées x, y avant d'étendre la logique à une liste de points pour réaliser un parcours défini dans le repère global Webots.

Mon premier test à été de déplacer le robot manuellement vers mon premier point cible. Dès que le robot entrait dans un rayon de tolérance autour de la cible, le programme indiquait un succès dans le terminal.

Pour réaliser une séquence de points cibles, j'ai fourni au programme une liste de coordonnées à atteindre avec une incrémentation de l'index à chaque succès du robot. Ces coordonnées ont dans un premier temps été obtenues à la main en déplaçant le robot sur le circuit. Très vite, je me suis rendu compte que ces coordonnées étaient faussées par le biais s'accumulant.

2.2 Changement de repère

Pour que le robot puisse s'orienter et se déplacer, il faut traduire la position de la cible dans son propre repère. Cela permet de calculer l'erreur angulaire et la distance euclidienne du robot à la cible.

Le calcul se fait en deux étapes :

- Rotation pour exprimer cette position relative par rapport à l'orientation actuelle du robot (θ) :
 - Calcul de l'angle absolu de la cible (α) : $\alpha = \text{atan2}(\Delta y, \Delta x)$
 - Calcul de l'erreur angulaire (ϵ_θ) : $\epsilon_\theta = \alpha - \theta$
- Position par le calcul de la distance euclidienne de la formule

$$\rho = \sqrt{\Delta x^2 + \Delta y^2}$$

Pour éviter que le robot ne fasse des tours sur lui-même inutilement, l'erreur est normalisée dans l'intervalle $[-\pi, \pi]$. Voici le code python permettant de réaliser ces calculs :

```
1 dx = target_x - x
2 dy = target_y - y
3 rho = math.sqrt(dx**2 + dy**2)
4 alpha = math.atan2(dy, dx)
5 angle_error = alpha - theta
6
7 # Normalisation de l'angle
8 while angle_error > math.pi: angle_error -= 2 * math.pi
9 while angle_error < -math.pi: angle_error += 2 * math.pi
```

2.3 Asservissement

L'asservissement implémenté pour diriger le robot vers la cible remplace le contrôle au clavier par une séquence de points de passage à rallier de manière autonome. L'algorithme ici calcule à chaque itération les commandes motrices nécessaires pour réduire l'erreur de position et d'orientation

L'asservissement est décomposé en une machine à états. C'est une approche très séquentielle et qui ne permet pas de courbe complexe comme on pourrait le voir avec une loi de commande par exemple.

- Correction d'orientation : Si l'erreur angulaire $|\epsilon_\theta|$ est supérieure à un seuil défini (ANGLE_THRESHOLD), le robot pivote sur place.
- Réduction de la distance : Une fois aligné avec la cible, le robot avance en ligne droite tant que la distance ρ est supérieure à DIST_THRESHOLD.

```
1 if distance > DIST_THRESHOLD:
2     if abs(angle_error) > ANGLE_THRESHOLD:
3         # Étape 1 : S'orienter
4         if angle_error > 0:
5             left_speed, right_speed = -V_ROT, V_ROT
6         else:
7             left_speed, right_speed = V_ROT, -V_ROT
8     else:
9         # Étape 2 : Avancer
10        left_speed, right_speed = V_TRANS, V_TRANS
11    else:
12        # Étape 3 : Point atteint, passer au suivant
13        print(f"Point {current_target_index + 1} atteint !")
14        current_target_index += 1
15        left_speed, right_speed = 0, 0
```

Pour palier au problème des points mesurés et contenant des erreurs de biais, j'ai repris des coordonnées arbitraire correspondant à mon circuit :

```
1 targets = [
```

```

2  (0.150, -0.620),
3  (-0.470, -0.625),
4  (-0.470, -0.400),
5  (-0.075, -0.400),
6  (-0.060, 0.600),
7  (0.150, 0.600),
8  (0.150, 0.120),
9  (0.550, 0.120),
10 (0.550, y),
11 (x, y) #point d'origine
12 ]

```

Je les ai ensuite affichées dans mon graphique où la prochaine cible est une croix dans un rond et les cibles suivantes sont une croix.

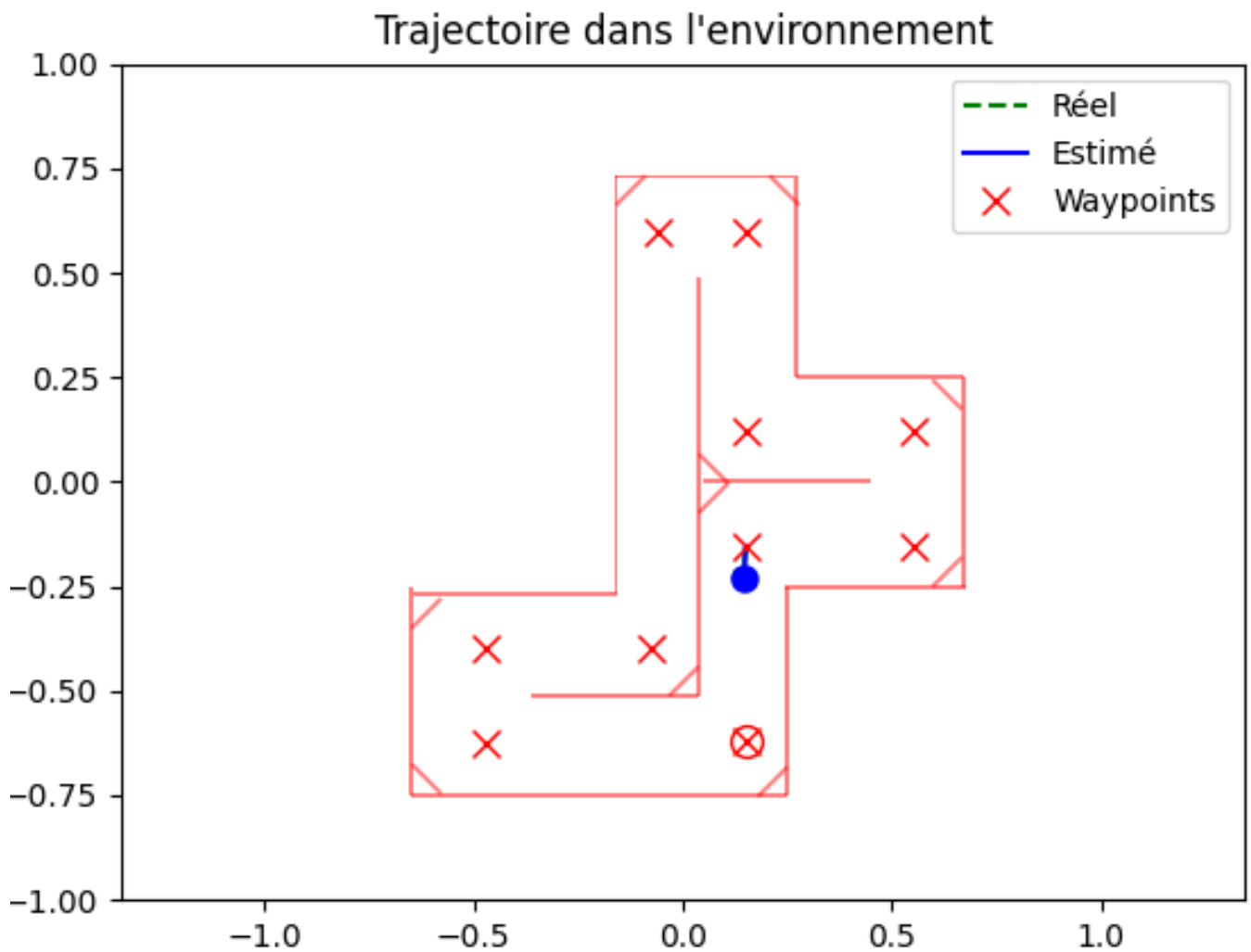


Figure 2.1 – Affichage de la séquence de cible dans le circuit

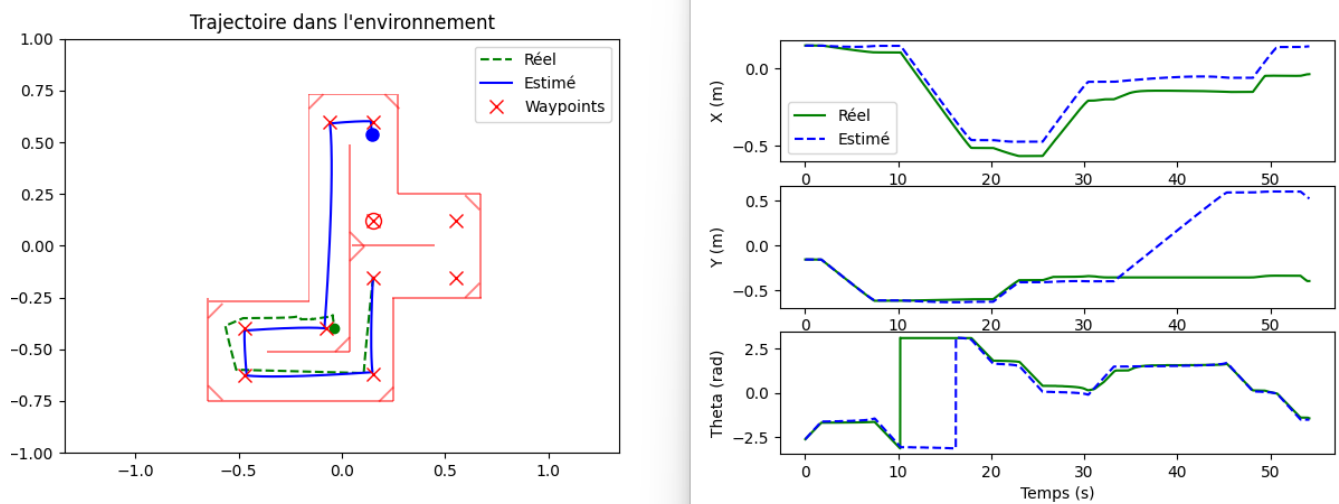


Figure 2.2 – Trajectoire automatique réalisée

Lorsque l'on exécute le programme, on peut voir que le robot peine à parcourir correctement le circuit : Cela vient du biais accumulé trop important et à certaines collisions dans un mur donnant de nouvelles erreurs de mesure. Contrairement aux courbes vues dans la partie 1, le déplacement du robot n'est pas géré par un opérateur ayant connaissance de la position réelle du robot. Ici, le manque de correction d'estimation empêche le bon fonctionnement du programme.

Pour corriger la trajectoire du robot, nous pourrions utiliser un système de géolocalisation au travers de balises stationnaires avec un filtre de kalman étendu (EKF) pour estimer puis corriger la position et l'orientation du robot.